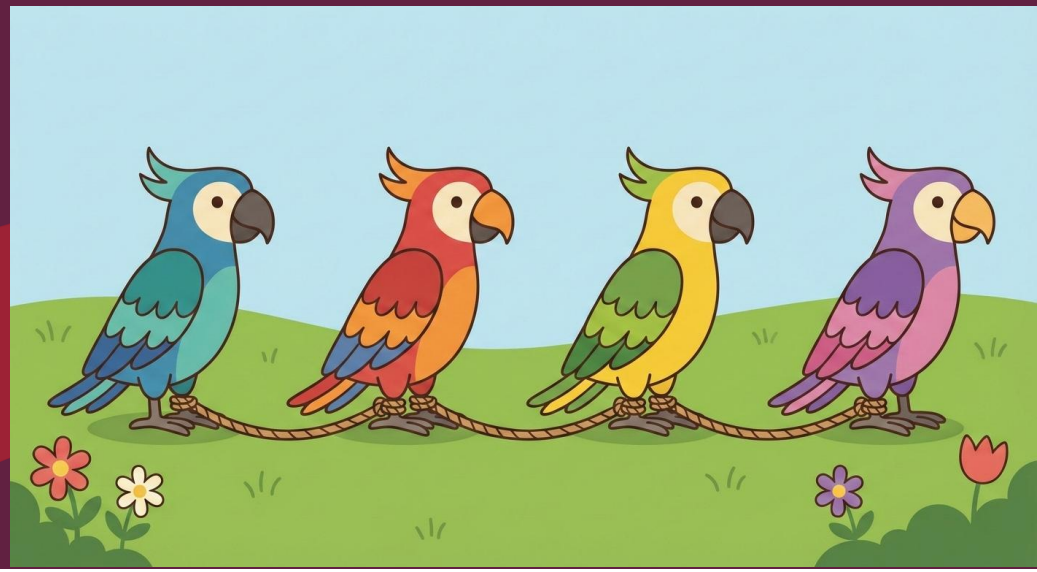
Agentic AI with LangChain and LangGraph



Presentation 4: Middleware
Primary Instructor: Michael Z
RA: YH
Date: 2026.Jan-Feb

Sobey School of Business
Saint Mary's University
MBAN

Saint Mary's University

0. Middleware

0.1 Middleware: Middleware is introduced by LangChain, not ReAct architecture

If you still remember what we discussed previously about ReAct, you can notice that ReAct has nothing to do with Middleware.

Middleware seems to be only available in LangChain and Microsoft Agents framework, so you won't learn this in other agents building platforms, at the time I was writing this slide (i.e. Dec 30th, 2025).

LangChain Blog
https://blog.langchain.com › agent-middleware

Agent Middleware

Sep 8, 2025 — A new **Middleware** abstraction we are introducing in LangChain 1.0 which we think makes it the most flexible and composable **agent** abstraction out ... Read more

Microsoft Learn
https://learn.microsoft.com › en-us › user-guide › agents

Agent Middleware

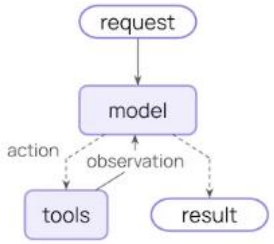
Dec 7, 2025 — **Middleware** in the **Agent** Framework provides a powerful way to intercept, modify, and enhance **agent** interactions at various stages of ... Read more

Agent Run Middleware | Agent Run Streaming...

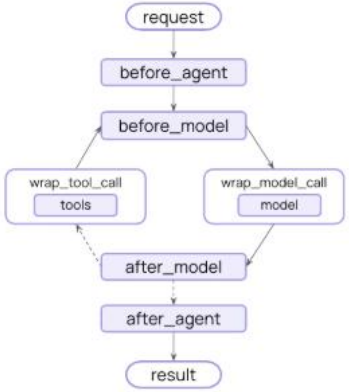
0.2 Middleware: Why we should learn Middleware and What does it mean?

Middleware can help you control the agent behavior further than regular configurations.

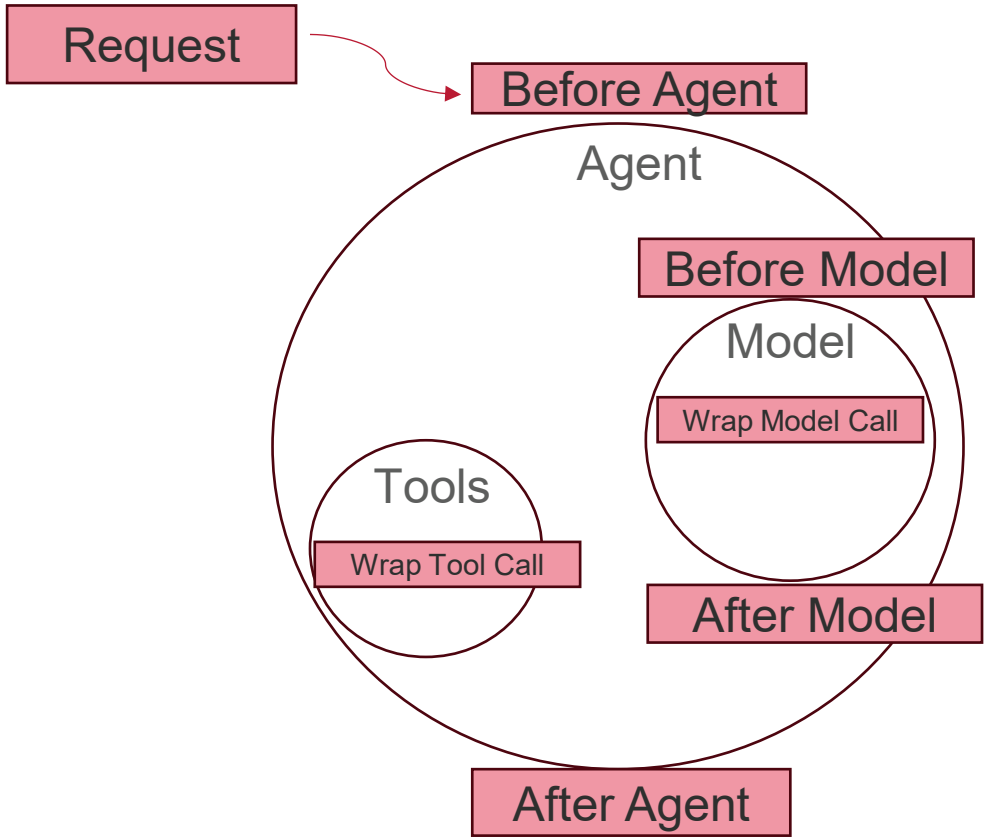
The core agent loop involves calling a model, letting it choose tools to execute, and then finishing when it calls no more tools:



Middleware exposes hooks before and after each of those steps:



Let me draw a graph for you to help you understand better.



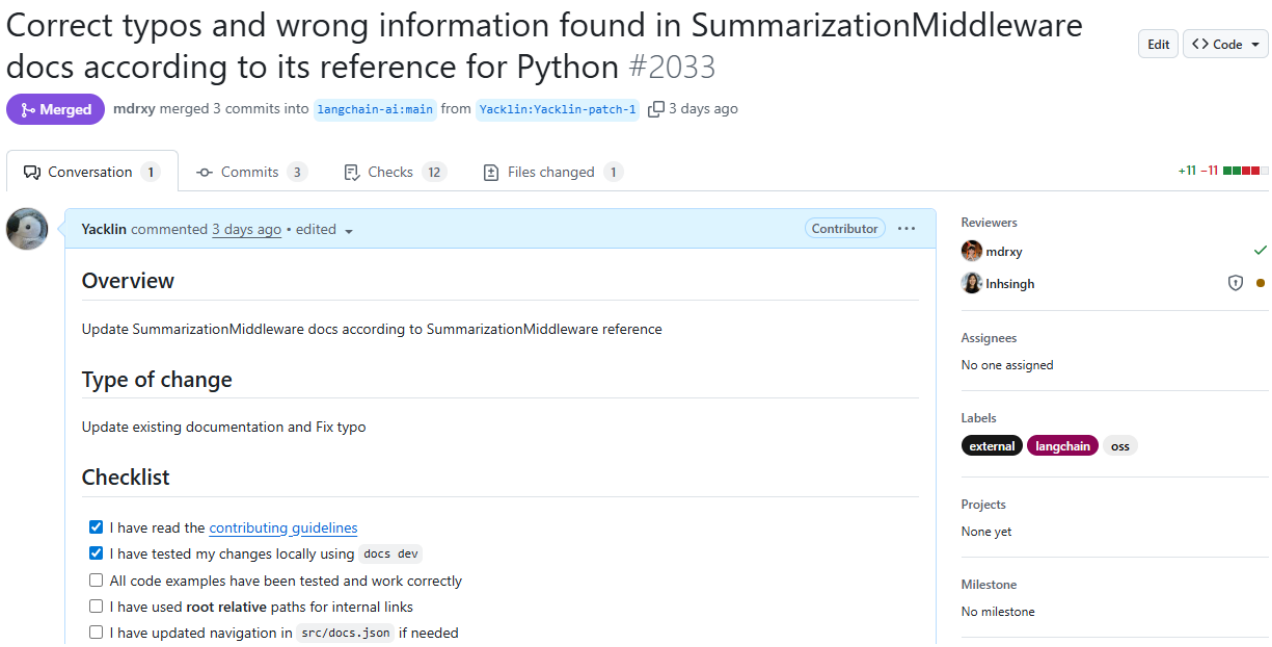
(Tbh, it's really hard to position wrap_model_call and wrap_tool_call 😭)

0.3 Middleware: Your learning outcome is to understand built-in Middleware

I think right now you are able to understand Middleware-related content we skipped at last presentation.

Also, I assume you are able to read the Built-in middleware section on your own right now.
I would go over some (or all) built-in middleware with you (I would jump to LangChain docs website. What's more, LangChain provides video tutorials and feel free to watch if you like!)

By the way, when preparing this slide, I noticed there are some issues in the documents of built-in Middleware, so I proposed a fix to LangChain, and they approved then merged my fix.



<u>But my fix has a typo, can you point it out?</u>

I think Michael would provide bonus to you for doing this.

Sobey School of Business
Saint Mary's University
MBAN

0.4 Middleware: Some Middleware are optimized for some vendors

Good to know…

Provider-specific middleware

These middleware are optimized for specific LLM providers. See each provider's documentation for full details and examples.

Anthropic

Prompt caching, bash tool, text editor, memory, and file search middleware for Claude models.

OpenAI

Content moderation middleware for OpenAI models.

1. Custom Middleware

1.1 Custom Middleware: Read their examples

Control agent's behavior in a way you like. You could control almost everything to meet your, or your employer's needs.

Read their examples to be comfortable with it.

Examples

- Dynamic model selection
- Tool call monitoring
- Dynamically selecting tools
- Working with system messages

1.2 Custom Middleware: Execution Order

Order of middleware being passed in matters.

These rules remind me of structure of Onion……
Hope it helps you understand!

```python
agent = create_agent(
    model="gpt-4o",
    middleware=[middleware1, middleware2, middleware3],
    tools=[...],
)
```

Execution flow

Before hooks run in order:

1. `middleware1.before_agent()`
2. `middleware2.before_agent()`
3. `middleware3.before_agent()`

Agent loop starts

4. `middleware1.before_model()`
5. `middleware2.before_model()`
6. `middleware3.before_model()`

Wrap hooks nest like function calls:

7. `middleware1.wrap_model_call()` → `middleware2.wrap_model_call()` → `middleware3.wrap_model_call()` → model

After hooks run in reverse order:

8. `middleware3.after_model()`
9. `middleware2.after_model()`
10. `middleware1.after_model()`

Agent loop ends

11. `middleware3.after_agent()`
12. `middleware2.after_agent()`
13. `middleware1.after_agent()`

1.2 Custom Middleware: Advice when playing with Middleware

In conclusion, make sure you understand what you are doing!

Best practices

1. Keep middleware focused - each should do one thing well
2. Handle errors gracefully - don't let middleware errors crash the agent
3. **Use appropriate hook types**:
   - Node-style for sequential logic (logging, validation)
   - Wrap-style for control flow (retry, fallback, caching)
4. Clearly document any custom state properties
5. Unit test middleware independently before integrating
6. Consider execution order - place critical middleware first in the list
7. Use built-in middleware when possible

Thank you for your paged attention!

arXiv > cs > arXiv:2309.06180

Computer Science > Machine Learning

[Submitted on 12 Sep 2023]

Efficient Memory Management for Large Language Model Serving with PagedAttention

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, Ion Stoica